

## 11.2-Logistic\_Regression\_MultiClass

August 4, 2025

## 1 Logistic Regression For Multiclass Classification

```
[1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: ## creating the dataset using sklearn library
from sklearn.datasets import make_classification
```

```
[3]: X, y = make_classification(n_samples = 1000, n_features = 10, n_informative=3,
    ↪ n_classes=3, random_state=15)
```

```
[4]: df = pd.DataFrame(X)
df.head()
```

|      |           |           |           |           |           |           |           |   |
|------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|---|
| [4]: | 0         | 1         | 2         | 3         | 4         | 5         | 6         | \ |
| 0    | -1.286132 | -0.648334 | 1.044115  | -0.469715 | 0.789859  | -0.083727 | -1.647309 |   |
| 1    | -0.222224 | 2.083232  | 1.191114  | -1.354527 | -0.956992 | 1.696028  | -1.070406 |   |
| 2    | -0.431963 | 0.375745  | -1.370334 | 0.819214  | 0.345243  | 1.389447  | -1.904130 |   |
| 3    | -0.912633 | 0.986988  | -0.690042 | 2.014628  | -0.442260 | 0.590347  | -1.819353 |   |
| 4    | -0.367056 | 1.667892  | 0.879172  | 2.214276  | 1.846338  | -0.894047 | 1.543838  |   |
|      | 7         | 8         | 9         |           |           |           |           |   |
| 0    | -1.316412 | 1.011910  | -0.898063 |           |           |           |           |   |
| 1    | 0.981403  | -1.628798 | 1.377594  |           |           |           |           |   |
| 2    | 1.292602  | 0.925545  | 0.232705  |           |           |           |           |   |
| 3    | 1.560938  | 0.823755  | 0.517762  |           |           |           |           |   |
| 4    | 0.931103  | -1.015210 | 1.061845  |           |           |           |           |   |

```
[5]: pd.DataFrame(y).value_counts()
```

```
[5]: 0
      2    335
      1    333
```

0 332  
Name: count, dtype: int64

```
[6]: from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LogisticRegression

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
      ↪random_state=15)
      logistic = LogisticRegression(multi_class='ovr') # Classification type is One v/
      ↪s rest
      logistic.fit(X_train, y_train)
      y_pred = logistic.predict(X_test)
```

```
[7]: y_pred
```

```
[7]: array([2, 0, 0, 0, 2, 0, 1, 1, 2, 0, 2, 1, 2, 1, 0, 0, 2, 2, 0, 2, 0, 0,
          0, 1, 1, 1, 1, 2, 2, 2, 1, 0, 2, 2, 2, 2, 1, 2, 2, 0, 2, 1, 0, 2,
          1, 1, 2, 0, 0, 0, 2, 0, 1, 2, 0, 2, 1, 2, 2, 1, 0, 0, 0, 2, 1, 2,
          1, 0, 2, 0, 2, 0, 2, 1, 2, 0, 2, 1, 2, 2, 1, 0, 0, 2, 2, 0, 2, 2,
          1, 0, 1, 0, 0, 0, 1, 1, 2, 1, 1, 0, 1, 1, 1, 2, 1, 1, 1, 0, 0, 1,
          2, 1, 2, 1, 0, 1, 2, 0, 0, 1, 0, 0, 2, 1, 0, 1, 0, 2, 2, 1, 0, 2,
          2, 2, 2, 2, 0, 2, 0, 2, 1, 0, 2, 0, 2, 2, 0, 1, 1, 2, 1, 2, 1, 1,
          0, 0, 1, 1, 1, 2, 2, 0, 0, 2, 2, 1, 2, 1, 1, 1, 0, 0, 1, 0, 1, 2,
          0, 2, 2, 0, 2, 1, 1, 1, 2, 1, 0, 1, 0, 1, 1, 2, 0, 2, 1, 2, 1, 0,
          1, 2, 0, 2, 1, 0, 1, 0, 2, 1, 0, 1, 2, 0, 0, 0, 0, 0, 0, 1, 2, 0,
          2, 0, 2, 1, 2, 1, 0, 0, 1, 1, 2, 1, 0, 2, 1, 0, 2, 1, 1, 2, 2, 1,
          1, 2, 2, 1, 2, 2, 2, 2, 0, 2, 2, 2, 0, 2, 0, 0, 1, 2, 1, 1, 2, 0,
          2, 0, 1, 0, 1, 2, 2, 1, 2, 2, 2, 2, 0, 2, 1, 0, 0, 0, 2, 1, 2, 2,
          2, 1, 0, 0, 2, 1, 2, 2, 2, 2, 1, 2, 1, 2])
```

```
[8]: from sklearn.metrics import accuracy_score, classification_report,
      ↪confusion_matrix

      acc = accuracy_score(y_test, y_pred)
      print("accuracy_score is: ", acc)
      com = confusion_matrix(y_test, y_pred)
      print("confusion_matrix is:\n", com)
      clf = classification_report(y_test, y_pred)
```

```
accuracy_score is: 0.7833333333333333
confusion_matrix is:
[[83 16  6]
 [ 2 69 26]
 [ 6  9 83]]
```

```
[9]: print(clf)
```

```
precision    recall  f1-score   support
```

|              |      |      |      |     |
|--------------|------|------|------|-----|
| 0            | 0.91 | 0.79 | 0.85 | 105 |
| 1            | 0.73 | 0.71 | 0.72 | 97  |
| 2            | 0.72 | 0.85 | 0.78 | 98  |
| accuracy     |      |      | 0.78 | 300 |
| macro avg    | 0.79 | 0.78 | 0.78 | 300 |
| weighted avg | 0.79 | 0.78 | 0.78 | 300 |

## 2 Work With Hyperparameter Tuning

### 2.1 GridSearchCV

```
[10]: model = LogisticRegression(multi_class='ovr')
```

```
[11]: params = [
    # Solvers that only support l2
    {'solver': ['newton-cg', 'lbfgs', 'sag'], 'penalty': ['l2'], 'C': [100, 10, 1, 0.1, 0.01]},

    # liblinear supports l1 and l2
    {'solver': ['liblinear'], 'penalty': ['l1', 'l2'], 'C': [100, 10, 1, 0.1, 0.01]},

    # saga supports all penalties
    {'solver': ['saga'], 'penalty': ['l1', 'l2', 'elasticnet'], 'C': [100, 10, 1, 0.1, 0.01]}
]
```

```
[12]: from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.linear_model import LogisticRegression

cv = StratifiedKFold(n_splits=5)
grid = GridSearchCV(estimator=model, param_grid=params, scoring='accuracy', cv=cv, n_jobs=-1)

# Fit the model
grid.fit(X_train, y_train)

# Get the best score
print(grid.best_score_)
```

```
0.7985714285714286
```

```
[13]: grid.best_params_
```

```
{'C': 0.1, 'penalty': 'l1', 'solver': 'liblinear'}
```

```
[14]: y_pred = grid.predict(X_test)

[15]: score = accuracy_score(y_pred, y_test)
print(score)
print('\n')
print("classification_report is:\n", classification_report(y_test, y_pred))
print("confusion_matrix is:\n", confusion_matrix(y_pred, y_test))
```

0.7866666666666666

classification\_report is:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.92      | 0.78   | 0.85     | 105     |
| 1            | 0.75      | 0.71   | 0.73     | 97      |
| 2            | 0.71      | 0.87   | 0.78     | 98      |
| accuracy     |           |        | 0.79     | 300     |
| macro avg    | 0.80      | 0.79   | 0.79     | 300     |
| weighted avg | 0.80      | 0.79   | 0.79     | 300     |

confusion\_matrix is:

```
[[82  2  5]
 [15 69  8]
 [ 8 26 85]]
```

Why the Scores Are Different:

grid.best\_score\_ = 80%: This is the average accuracy across 5-fold cross-validation on the training set only. It's used to find the best hyperparameters.

classification\_report = 79%: After GridSearchCV found the best model, you tested it on new, unseen test data — naturally, performance can drop slightly here.

## 2.2 Randomized SearchCV

```
[16]: from sklearn.model_selection import RandomizedSearchCV
ranadmcv = RandomizedSearchCV(estimator=model, param_distributions=params,
                               scoring='accuracy', cv=10, n_jobs=-1)
```

```
[17]: ranadmcv.fit(X_train, y_train)
```

```
[17]: RandomizedSearchCV(cv=10, estimator=LogisticRegression(multi_class='ovr'),
                        n_jobs=-1,
                        param_distributions=[{'C': [100, 10, 1, 0.1, 0.01],
                                             'penalty': ['l2'],
                                             'solver': ['newton-cg', 'lbfgs',
                                                         'sag']}],
```

```
{'C': [100, 10, 1, 0.1, 0.01],
  'penalty': ['l1', 'l2'],
  'solver': ['liblinear']},
{'C': [100, 10, 1, 0.1, 0.01],
  'penalty': ['l1', 'l2', 'elasticnet'],
  'solver': ['saga']}]},
```

```
scoring='accuracy')
```

```
[18]: ranadmcv.best_estimator_
```

```
[18]: LogisticRegression(C=0.1, multi_class='ovr', penalty='l1', solver='saga')
```

```
[19]: ranadmcv.best_params_
```

```
[19]: {'solver': 'saga', 'penalty': 'l1', 'C': 0.1}
```

```
[20]: y_pred = ranadmcv.predict(X_test)
```

```
[21]: ranadmcv.best_score_
```

```
[21]: 0.7942857142857143
```

```
[22]: score = accuracy_score(y_pred, y_test)
print(score)
print('\n')
print("classification_report is:\n", classification_report(y_test, y_pred))
print("confusion_matrix is:\n", confusion_matrix(y_pred, y_test))
```

```
0.7833333333333333
```

```
classification_report is:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.91      | 0.78   | 0.84     | 105     |
| 1            | 0.75      | 0.71   | 0.73     | 97      |
| 2            | 0.71      | 0.86   | 0.78     | 98      |
| accuracy     |           |        | 0.78     | 300     |
| macro avg    | 0.79      | 0.78   | 0.78     | 300     |
| weighted avg | 0.79      | 0.78   | 0.78     | 300     |

```
confusion_matrix is:
```

```
[[82  2  6]
 [15 69  8]
 [ 8 26 84]]
```

```
Same as GridSearchCV
```

```
[26]: plt.figure(figsize=(10, 6))
sns.scatterplot(x=pd.DataFrame(X)[0], y=pd.DataFrame(X)[1], hue=y )
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Data Distribution by Class')
plt.legend(title='Class')
plt.show()
```

